Nombre (s) :	Matrícula (s):
Darío Alberto Sánchez Perzabal	A01738266
Santiago Patricio Gómez Ochoa	A01735171
Ximena Ortiz Gómez	A01735100
María José Croche Álvarez	A01734561

Nombre del curso:	Nombre del profesor (es):
Implementación de Robótica Inteligente	Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez Cesar Torres Huitzil
Módulo:	**Actividad:** Actividad R_2. Cálculo del error de un robot móvil diferencial
Fecha: 17 de abril del 2026	

Resumen

Esta actividad busca un mejor entendimiento del lazo cerrado, esto por medio del comportamiento del robot Puzzlebot, utilizando un método de integración numérico para obtener las velocidades lineales y angulares, base para el movimiento de un robot diferencial. Además es necesario tener conocimientos sobre la estructura del mensaje */odom*, pues su salida forma parte de la retroalimentación del control de lazo cerrado. Finalmente, en ROS2 es importante tener en cuenta la *Quality of Service*, ya que define y configura la comunicación entre nodos.

Objetivos

El objetivo principal de la actividad es comprender el lazo cerrado en el puzzlebot, para ello primeramente es necesario entender la implementación del método de integración numérico de Euler, pues es necesario para obtener las velocidades lineales y angulares. Igualmente, es necesario comprender los valores del error, y la estructura del mensaje */odom,* el cual recibe las velocidades angulares de ambos motores, se realiza el cálculo en el nodo y */odom* publica las velocidades lineales y angulares, que sirve como retroalimentación para el lazo cerrado.

Introducción

En el desarrollo de robótica móvil uno de los principales retos es el lograr que el robot sea capaz de conocer su posición, orientarse en un entorno o espacio y desplazarse hacia un objetivo de una manera controlada. Es por ello que es fundamental conocer y comprender conceptos como lo es la odometría, la navegación y el cálculo de errores de posición, los cuales permiten interpretar el movimiento del robot y tomar decisiones en tiempo real de acuerdo a las funciones del sistema.

Por ende, resulta importante saber que es la odometría. La odometría es el proceso en el cual los robots pueden actualizar su posición en el tiempo, siempre que sepan donde partieron [1]. En el caso del Puzzlebot, el robot móvil utilizado en estas actividades, la odometría consiste en estimar la posición y orientación del robot a partir de los datos obtenidos de los encoders de los motores de las ruedas. Estos sensores permiten que se pueda modelar el movimiento del robot a partir de las velocidades de cada rueda y nos permite estimar su velocidad lineal, angular y su posición en el plano o el espacio. No obstante, algo a tener en cuenta es que la odometría puede presentar errores acumulativos debido a factores externos como el deslizamiento de las ruedas.

De igual forma, un elemento importante de esta actividad es el cálculo del error el cual permite que se pueda evaluar la diferencia entre la posición actual del robot y una posición objetivo. Es por ello que para la estimación del error se calcula a través de la diferencia entre la orientación del robot y dirección hacia el objetivo. Este cálculo es fundamental para poder implementar un control de lazo cerrado que nos permita tener un sistema controlado y con retroalimentación.

Consecuentemente, para la implementación de estos conceptos en ROS se utiliza un mensaje de tipo Odometry parte del paquete *nav_msgs* el cual nos permite guardar y enviar información relevante como la posición, la orientación y las velocidades del Puzzlebot con el fin de que distintos nodos puedan acceder a la información del estado del robot de manera estructurada y en tiempo real [2].

Finalmente resulta importante definir el concepto de QoS o conocidos mejor como Quality of Service, el cual define y permite configurar la comunicación entre nodos en ROS2, así como su fiabilidad, durabilidad y el historial de entrega de mensajes [3]. Estas configuraciones son esenciales para realizar odometría del robot dado que garantizan que la información crítica se transmita de manera adecuada dependiendo de los requerimientos del sistema. Es por ello que a lo largo de esta actividad y el reto semanal 3 se integran conceptos fundamentales que permitirán desarrollar nodos que estimen velocidades y posiciones del

robot mediante odometría y un control de lazo cerrado, algo fundamental para aplicaciones robóticas reales y actuales.

Solución del problema

Actividad 2.1

La actividad **2.1** tiene como objetivo principal integrar las velocidades angulares y lineales, esto por el método de Euler, el cual es una técnica numérica, el cual es utilizado para aproximar soluciones, en este caso se estima la posición del robot. Para aproximar la siguiente posición, se basa en el estado actual del robot, las velocidades lineales y angulares, y un delta de tiempo, es decir, un pequeño intervalo de tiempo.

Es importante mencionar que este método de integración, computacionalmente es bastante eficiente, lo que permite ser ejecutado en tiempo real, además es un método sencillo que permite tener en cuenta las restricciones no holónomas que tiene el robot Puzzlebot.

¿La trayectoria es uniforme o se desvía?, si se desvía, ¿por qué lo hace?

La trayectoria es uniforme, sin embargo, no hace un cuadrado perfecto. Esto puede deberse a diferentes factores, el primero siendo el controlador P que tiene implementado, para ello debe encontrarse una mejor ganancia para *kp,* igualmente puede deberse a la integral por el método de euler, ya que como se mencionó anteriormente, se hace una aproximación, no es completamente exacta, por lo que hay márgenes de error en el cálculo y por consecuencia en la trayectoria final. Es importante mencionar que tanto el ángulo como la distancia tienen una tolerancia, esto para evitar también comportamientos no deseados.

Actividad 3.1

Para la implementación del cálculo del error en el robot móvil diferencial Puzzlebot se trabajó con los dos nodos principales del reto puesto que ya se habían realizado con anterioridad los cuales son los nodos de odometría y un nodo controlador. El nodo odometría se encarga de estimar la posición y orientación del robot a partir de las velocidades de las ruedas y publicar esta información mediante un mensaje tipo Odometry. A partir de estos datos, el nodo controlador se subscribe al tópico /odom y obtiene en tiempo real la pose actual del robot, es decir su posición y orientación. Con estos datos dentro del mismo nodo se realiza el cálculo del error con respecto a una posición objetivo.

Hablando específicamente del cálculo del error , el cálculo del error se lleva a cabo en dos partes. Primeramente se obtiene el error de distancia, que representa qué tan lejos está el robot del objetivo en cuestión de distancia. Posteriormente se calcula el error angular,

el cual corresponde a la diferencia entre la orientación actual del robot y la dirección hacia el punto objetivo. Para ello, en el código se calculan las variables dx y dy como la diferencia entre la posición deseada y la posición actual del robot. Posteriormente, se obtiene distance_error mediante la distancia euclidiana y theta_error a partir del ángulo deseado y la orientación actual del Puzzlebot. Finalmente, este error angular se normaliza para mantenerlo dentro de un rango deseado que en este caso es un círculo.

Consecuentemente, el nodo controller utiliza estos valores calculados para definir qué movimiento hará el robot, por ende cuando el error angular es grande, el robot gira en su propio eje para alinearse para llegar al objetivo deseado, mientras que cuando el error angular es pequeño el robot avanza mientras sigue corrigiendo su orientación. Por ende, el error sirve como base para el seguimiento de una trayectoria.

Resultados

Como se mencionó con anterioridad, para estas actividades se implementó un nodo de odometría encargado de estimar la posición del robot en el plano a partir de las velocidades de las ruedas. Es importante mencionar que el cálculo de las velocidades lineal y angular (correspondiente a la actividad 1.1) se integró dentro de este mismo nodo, ya que es un paso necesario para poder realizar la integración de la posición.

Durante la ejecución de nuestro programa, el nodo de la odometría calcula la velocidad lineal y angular del robot a partir de las velocidades de las ruedas y posteriormente integra estos valores mediante el método de Euler para obtener la posición del robot. Por ende contestando a las preguntas de la actividad 1.1 podemos observar que cuando ambas ruedas presentan velocidades similares, el robot se desplaza en línea recta con un poco de oscilaciones, mientras que cuando existe diferencias entre las velocidades de las ruedas el robot realiza trayectorias curvas, giros o vueltas.

Consecuentemente, al realizar la actividad 2.1 se pudo observar la trayectoria del robot fue visualizada en tiempo real mediante Matplotlib mostrando el recorrido en el espacio. En la figura 1 se observa que el robot sigue la trayectoria cuadrada correspondiente a los puntos definidos en el controlador. No obstante, la trayectoria no es perfecta ya que presenta oscilaciones debido a aproximaciones del método Euler, errores acumulados en la odometría o limitaciones en el controlador implementado.

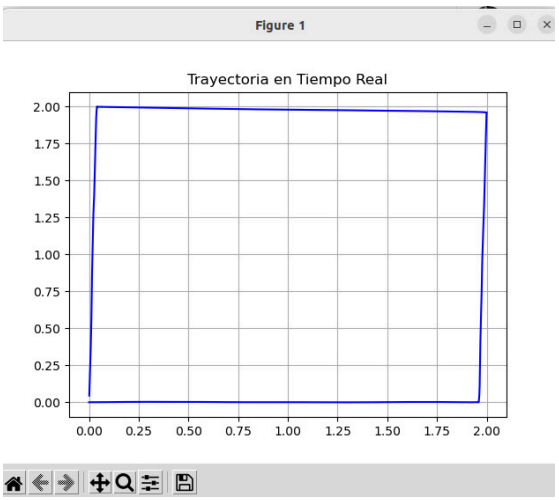

Imagen 1. Trayectoria en tiempo real del robot con odometría

Analizando la imagen 1, podemos observar que la trayectoria es mayormente uniforme, pero presenta ligeras desviaciones debido a errores en la aproximación numérica, incertidumbre en la medición de velocidades y a que el controlador proporcional no corrige completamente el error, es por ello que el controlador y sus variables deben de irse tuneando para llegar al punto en el que estas desviaciones sean las menos posibles.

Consecuentemente, durante la ejecución del nodo del controlador, se visualizaron en tiempo real la velocidad angular, la velocidad lineal, la posición en x y en y del valor del error de la trayectoria del robot como se muestra en la figura 2.

```
> Error: dist=0.095, angle=-0.000
> Comandos: v=0.076, w=-0.000
----------------------------------------
[INFO] [1776364616.386643062] [controller_sqr]: Entrando a control_loop
[INFO] [1776364616.417675203] [controller_sqr]: Avanzando -> v=0.07, w=-0.00
[INFO] [1776364616.418135099] [controller_sqr]: Comando publicado en /cmd_vel
[INFO] [1776364616.418484201] [controller_sqr]:
> Posición: x=0.002, y=0.086, theta=-1.594
> Error: dist=0.086, angle=-0.000
> Comandos: v=0.069, w=-0.001
----------------------------------------
[INFO] [1776364616.486121425] [controller_sqr]: Entrando a control_loop
[INFO] [1776364616.513059677] [controller_sqr]: Avanzando -> v=0.06, w=-0.00
[INFO] [1776364616.513363351] [controller_sqr]: Comando publicado en /cmd_vel
[INFO] [1776364616.513518611] [controller_sqr]:
> Posición: x=0.002, y=0.079, theta=-1.594
> Error: dist=0.079, angle=-0.000
> Comandos: v=0.063, w=-0.001
----------------------------------------
[INFO] [1776364616.586274691] [controller_sqr]: Entrando a control_loop
[INFO] [1776364616.616163470] [controller_sqr]: Avanzando -> v=0.06, w=0.00
[INFO] [1776364616.616436160] [controller_sqr]: Comando publicado en /cmd_vel
[INFO] [1776364616.616582115] [controller_sqr]:
> Posición: x=0.002, y=0.072, theta=-1.596
> Error: dist=0.072, angle=0.001
> Comandos: v=0.058, w=0.003
----------------------------------------
[INFO] [1776364616.686702334] [controller_sqr]: Entrando a control_loop
[INFO] [1776364616.717384798] [controller_sqr]: Avanzando -> v=0.05, w=-0.00
[INFO] [1776364616.717681462] [controller_sqr]: Comando publicado en /cmd_vel
[INFO] [1776364616.717829432] [controller_sqr]:
> Posición: x=0.002, y=0.065, theta=-1.594
> Error: dist=0.065, angle=-0.001
> Comandos: v=0.052, w=-0.001
```

Imagen 2. Mensaje /odom que publica las posiciones, el error y las velocidades cuando el robot avanza en línea recta

```
----------------------------------------
[INFO] [1776364612.186200217] [controller_sqr]: Entrando a control_loop
[INFO] [1776364612.216584303] [controller_sqr]: Avanzando -> v=0.30, w=0.02
[INFO] [1776364612.216840783] [controller_sqr]: Comando publicado en /cmd_vel
[INFO] [1776364612.216988080] [controller_sqr]:
> Posición: x=0.018, y=1.137, theta=-1.596
> Error: dist=1.137, angle=0.009
> Comandos: v=0.300, w=0.018
----------------------------------------
[INFO] [1776364612.286780083] [controller_sqr]: Entrando a control_loop
[INFO] [1776364612.320199841] [controller_sqr]: Avanzando -> v=0.30, w=0.02
[INFO] [1776364612.320525029] [controller_sqr]: Comando publicado en /cmd_vel
[INFO] [1776364612.320677737] [controller_sqr]:
> Posición: x=0.017, y=1.107, theta=-1.596
> Error: dist=1.107, angle=0.009
> Comandos: v=0.300, w=0.018
----------------------------------------
[INFO] [1776364612.386840082] [controller_sqr]: Entrando a control_loop
[INFO] [1776364612.417871634] [controller_sqr]: Avanzando -> v=0.30, w=0.02
[INFO] [1776364612.418128729] [controller_sqr]: Comando publicado en /cmd_vel
[INFO] [1776364612.418289942] [controller_sqr]:
> Posición: x=0.017, y=1.077, theta=-1.596
> Error: dist=1.077, angle=0.009
> Comandos: v=0.300, w=0.019
----------------------------------------
[INFO] [1776364612.486540633] [controller_sqr]: Entrando a control_loop
[INFO] [1776364612.517312690] [controller_sqr]: Avanzando -> v=0.30, w=0.02
[INFO] [1776364612.517574668] [controller_sqr]: Comando publicado en /cmd_vel
[INFO] [1776364612.517731783] [controller_sqr]:
> Posición: x=0.016, y=1.048, theta=-1.598
> Error: dist=1.048, angle=0.012
> Comandos: v=0.300, w=0.024
----------------------------------------
```

Imagen 3. Mensaje /odom que publica las posiciones, el error y las velocidades cuando el robot avanza está girando

A partir de las imágenes y evidencias anteriores podemos observar que el error de distancia disminuye progresivamente cuando el error se acerca al objetivo. De igual manera, el error angular tiende a cero cuando el robot se encuentra correctamente orientado y también podemos observar que los comandos de la velocidad disminuyen conforme el robot se aproxima al objetivo. Lo anterior indica que el robot logra acercarse al objetivo de manera estable. Por ende podemos decir que mediante la ejecución del nodo controlador desarrollado, el cual genera automáticamente los comandos hacia los puntos objetivo se observó que cuando el error angular es grande, el robot gira para alinearse. Además, cuando el error angular es pequeño, el robot avanza y por ende, el error cambia conforme el robot se mueve. Esto confirma que el cálculo del error se realiza correctamente y que el sistema implementado para estas actividades responde adecuadamente.

Conclusiones

La actividad fue realizada exitosamente, pues cumplió con los objetivos principales, ya que como se pide en la actividad, el método de integración que se implementó de Euler, permitiendo que se hagan estimaciones de la posición del robot en tiempo real, posibilitando graficar la trayectoria también en tiempo real. Igualmente, se utilizó */odom,* mensaje que publica la odometría del robot, siendo la herramienta clave para la retroalimentación del control de lazo cerrado. Finalmente, hubo un mejor entendimiento de la importancia de *QoS,* fundamental a la hora de comunicar nodos implementados en ROS2.

Referencias

1. ROS Mobile Robots. (s.f.). *Odometría*. https://ros-mobile-robots.com/theory/modeling-control/odometry/
2. ROS Org. (s.f.). *Publishing Odometry Information over ROS*. https://wiki.ros.org/navigation/Tutorials/RobotSetup/Odom
3. Open Robotics (2026). *Quality of Service settings.* https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Quality-of-Service-Settings.html